

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 15

Тема: «Разработка комплексных тестовых сценариев (Unit + Integration
+ Contract) Язык программирования: Python 3.10+»

Цель работы:

Разработать набор тестов (модульных, интеграционных и контрактных) для микросервиса «**Система управления заказами и доставкой**», обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).

Выполнила: Чичерина

Диана Сергеевна,

Студент 24-ИС

2026г.

Вариант 11

ДОПУСК К ПРАКТИЧЕСКОЙ ЧАСТИ (УСТНЫЙ ОПРОС)

1. **pytest** — проще и мощнее unittest, пишется с assert, не требует классов.
2. **Фикстуры** — функции с `@pytest.fixture`, подготавливают данные для тестов, убирают дублирование.
3. **Мокать HTTP** — через `requests-mock` или `patch('requests.get')`, подменяем ответ без реального запроса.
4. **`@pytest.mark.parametrize`** — запускает один тест с разными параметрами (таблица данных).
5. **Проверка исключения** — `pytest.raises(Ошибка, match="текст")`.
6. **`Mock()`** — создает заглушку. **`patch()`** — подменяет реальный объект на заглушку.
7. **Асинхронные тесты** — `pytest-asyncio` + `@pytest.mark.asyncio` + `await`.
8. **Фабрика фикстур** — функция, создающая другие функции для генерации объектов с разными параметрами.
9. **Проверка вызовов** — `mock.assert_called_with(arg)`, `mock.assert_called_once()`.
10. **`coverage.py`** — показывает % покрытия тестами. Хороший порог — **80%+**.

```
project/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── services.py
│   ├── exceptions.py
│   └── routers/
│       └── orders.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   └── test_global_handler.py
├── requirements.txt
└── pytest.ini
```

```
===== warnings summary =====
c:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform win32, python 3.13.9-final-0
Coverage HTML written to dir htmlcov
===== 6 passed, 1 warning in 2.35s =====
```

Предупреждение:

StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.

Что было сделано и вывод:

На занятии разработана система тестирования для микросервиса на FastAPI. Создана структура проекта, реализовано REST API с эндпоинтом получения заказа и глобальный обработчик ошибок.

Написано 6 модульных тестов (pytest) с использованием моков и TestClient, проверяющих успешные сценарии. Запущены тесты с покрытием — результат 94% (выше порога 80%). Изучены принципы тестирования по паттерну Arrange-Act-Assert, использование моков, глобальные обработчики ошибок и анализ покрытия кода. Полученная архитектура соответствует SOLID, легко тестируется и расширяется.

Разработала набор тестов (модульных, интеграционных и контрактных) для микросервиса «**Система управления заказами и доставкой**», обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).